

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..5

WORKING WITH ENDPOINTS

 Copy as Markdown

Working with endpoints

Endpoints are how your app talks to your server: you write a method on the server, and Serverpod generates a typed Dart method your app calls directly, with no REST layer, JSON parsing, or API contract to maintain. This page covers defining endpoints, calling them from your app, and pointing the app at each environment. The rest of this section builds on it, page by page: [sessions](#), [error handling](#), [streaming](#), [server events](#), and [file uploads](#). See [Related](#) for the full map.

For the client code to be generated:

- Place the endpoint file anywhere under the `lib` directory of your server.
- Create a class that extends `Endpoint`.
- Define methods that return a typed `Future` (or a `Stream`, see [Streaming](#)) and take a `Session` object as their first argument.

The `Session` object holds information about the call being made and provides access to the database.

Create an endpoint

```
import 'package:serverpod/serverpod.dart';

class ExampleEndpoint extends Endpoint {
  Future<String> hello(Session session, String name) async {
    return 'Hello $name';
  }
}
```

The above code creates an endpoint called `example` (the `Endpoint` suffix is removed) with the `hello` method. While `serverpod start` is running, the client-side code is generated automatically with each save. Outside a start session, run `serverpod generate` (or `serverpod generate --watch`) in the server directory.



Ask AI

Call an endpoint from your app

Your app calls the method through the generated client:

```
var result = await client.example.hello('World');
```

The scaffolded Flutter app already creates that client in `lib/client.dart`, connected to your development server, and `main()` initializes it before running the app:

```
// lib/client.dart
final serverUrl = getServerUrl();

late final Client client;

Future<void> initializeClient() async {
  client = Client(await serverUrl)
    ..connectivityMonitor = FlutterConnectivityMonitor();
}

// lib/main.dart
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await initializeClient();
  runApp(const MyApp());
}
```

The `getServerUrl()` helper from `serverpod_flutter` picks the server address from the first of these that is set:

1. The `SERVER_URL` compile-time define, for [choosing the URL per build](#).
2. The `apiUrl` value in `assets/config.json`.
3. The default, `http://localhost:8080/`, where `localhost` adapts to the platform, so an Android emulator reaches your machine through `10.0.2.2` without any changes.

The `connectivityMonitor` lets the client observe network changes. Projects created with authentication also attach an `authSessionManager` here.

On a physical device, `localhost` is the device itself, so the app needs your machine's network address instead:

1. Find your machine's LAN IP, with `ifconfig` on Linux/macOS or `ipconfig` on Windows.
2. Set it as the server URL, either as `apiUrl` in `assets/config.json` or with `--dart-define=SERVER_URL=http://192.168.1.20:8080/`.

Also set `publicHost` in the development config to the same IP, so URLs the server hands out, such as public file links, point somewhere the device can reach; see [Configuration](#).

To enable browser credentials for CORS or use platform-native HTTP clients, see [Configure HTTP calls](#).

Point the client at each environment

The resolved server URL is the server the app connects to, so it changes between development, staging, and production. Keep the scaffolded client setup and choose the URL per build with `--dart-define`:

```
flutter run --dart-define=SERVER_URL=https://staging.example.com/
```

Without the define, the app falls back to `assets/config.json` and then the local server. For a Flutter web app served by your Serverpod web server, the server provides `config.json` at runtime with its own API address, so the deployed web app finds the right server without a rebuild. In production, use your deployed server's public URL. If you deploy to [Serverpod Cloud](#), that is the URL of your deployed app.

Restrict access to your endpoints

Endpoints are open to any client by default. To require a signed-in user, override `requireLogin` and to require specific scopes, override `requiredScopes`:

```
class PrivateEndpoint extends Endpoint {  
  @override  
  bool get requireLogin ⇒ true;  
  
  Future<String> onlyForSignedInUsers(Session session) async {  
    return 'Hello ${session.authenticated?.userIdentifier}';  
  }  
}
```

If a call fails the check, the app receives a typed error; see [error handling](#).

For how users sign in and how scopes work, see [Authentication](#).

Pass and return data

Parameters and return values can be of type `bool`, `int`, `double`, `String`, `UuidValue`, `Duration`, `DateTime`, `ByteData`, `Uri`, `BigInt`, `dynamic`, vector and geography types, or generated serializable objects ([data models](#)). You can also use `List`, `Map`, `Record`, and `Set`, strictly typed with the types above. Null safety is supported, and return types follow the same rules as parameters. A `DateTime` is always converted to UTC when passed.

Parameters are sent by name in the request between app and server, so renaming an endpoint method's parameters breaks older app builds. See [backward compatibility](#) before changing a published API.

Binary data over a method call is capped by the request size limit, 512 KiB by default (an oversized call fails with an HTTP 413 error). The limit is the `maxRequestSize` key in the [Configuration reference](#). For transferring files, use the [file upload](#) interface instead.

Exclude an endpoint from generation

If you want the code generator to ignore an endpoint definition, annotate either the entire class or individual methods with `@doNotGenerate`. This is useful for keeping a definition in your codebase without generating server or client bindings for it.

Exclude an entire `Endpoint` class

```
import 'package:serverpod/serverpod.dart';

@doNotGenerate
class ExampleEndpoint extends Endpoint {
  Future<String> hello(Session session, String name) async {
    return 'Hello $name';
  }
}
```

The above code generates no server or client bindings for the example endpoint.

Exclude individual `Endpoint` methods

```
import 'package:serverpod/serverpod.dart';

class ExampleEndpoint extends Endpoint {
  Future<String> hello(Session session, String name) async {
    return 'Hello $name';
  }

  @doNotGenerate
  Future<String> goodbye(Session session, String name) async {
    return 'Bye $name';
  }
}
```

In this case the `ExampleEndpoint` only exposes the `hello` method, and the `goodbye` method is not accessible externally. For how the annotation interacts with inheritance, see [Endpoint inheritance](#).

Test your endpoints

The generated test tools call your endpoints the same way production code does, against a test database. See [Get started with testing](#).

Related

- [Sessions](#): the object every endpoint method receives.
- [Error handling and exceptions](#): typed errors between server and app.
- [Streaming](#): push live data to your app.
- [File uploads](#): direct-to-storage uploads.
- [Custom cloud storage](#): local disk, NAS, and custom backends.
- [Endpoint inheritance](#): share behavior across endpoints and reshape module endpoints.
- [Server events](#): publish and subscribe to messages across sessions and servers.
- [Endpoint middleware](#): wrap every API request for logging or rate limiting.
- [Configure HTTP calls](#): CORS credentials and native HTTP stacks.
- [Working with models](#): define the data your endpoints exchange.